

Universidade de Brasília
Faculdade de Tecnologia

Cálculo Numérico Aplicado

Programa para Casa #6 (PPC6)

Prof. Dr. Rafael Gabler Gontijo

Mateus Leal Silva

Matrícula: 221028134

Brasília, 05 de julho de 2026

Sumário

1	Introdução	2
2	Formulação matemática e discretização	2
3	Métodos numéricos implementados	3
3.1	Eliminação de Gauss	3
3.2	Método de Liebmann	3
3.3	Liebmann com relaxação	3
4	Solução analítica unidimensional de comparação	3
5	Arquivos de saída gerados pelo programa	4
6	Caso numérico de demonstração	4
7	Resultados dos métodos	4
8	Campo de temperatura obtido	5
9	Comparação com a solução analítica 1D	6
10	Estudo do fator de relaxação	7
11	Estudo de refinamento de malha	8
12	Discussão dos resultados	8
13	Conclusão	8
14	Código-fonte Python	10

1 Introdução

Este relatório apresenta o desenvolvimento de um programa computacional para resolver o problema bidimensional de condução de calor em uma aleta retangular de seção transversal constante. A base da aleta possui temperatura prescrita, enquanto as demais superfícies trocam calor por convecção com o ambiente.

O problema é governado pela equação de Laplace:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0, \quad (1)$$

com condição de Dirichlet na base e condição de Robin nas superfícies expostas:

$$T(0, y) = T_b, \quad (2)$$

$$-k \frac{\partial T}{\partial n} = h(T - T_\infty). \quad (3)$$

O programa implementado gera automaticamente a malha, monta o sistema linear por diferenças finitas, resolve o problema por eliminação de Gauss, pelo método de Liebmann e pelo método de Liebmann com relaxação. Além disso, compara a solução bidimensional com a solução analítica unidimensional clássica de aletas.

2 Formulação matemática e discretização

Para uma malha uniforme com n_x nós na direção x e n_y nós na direção y , os espaçamentos são:

$$\Delta x = \frac{L}{n_x - 1}, \quad \Delta y = \frac{H}{n_y - 1}. \quad (4)$$

Para nós internos, a equação de Laplace é aproximada por diferenças centradas:

$$\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2} = 0. \quad (5)$$

Definindo:

$$a_x = \frac{1}{\Delta x^2}, \quad a_y = \frac{1}{\Delta y^2}, \quad (6)$$

a equação interna pode ser escrita como:

$$(2a_x + 2a_y)T_P - a_x T_E - a_x T_W - a_y T_N - a_y T_S = 0. \quad (7)$$

Nos contornos convectivos, aplica-se a condição de Robin. Na extremidade livre, por exemplo:

$$-k \frac{\partial T}{\partial x} = h(T - T_\infty). \quad (8)$$

Usando nó fictício e diferenças centradas:

$$T_E = T_W - 2\Delta x \frac{h}{k} (T_P - T_\infty). \quad (9)$$

Substituindo na equação de Laplace, obtém-se:

$$\left(2a_x + 2a_y + 2\frac{h}{k\Delta x} \right) T_P - 2a_x T_W - a_y T_N - a_y T_S = 2\frac{h}{k\Delta x} T_\infty. \quad (10)$$

Expressões análogas são aplicadas às superfícies superior e inferior. Nos cantos convectivos, duas condições de Robin são aplicadas simultaneamente.

3 Métodos numéricos implementados

3.1 Eliminação de Gauss

O primeiro método implementado resolve diretamente o sistema linear $[A]\{T\} = \{b\}$. A matriz é montada de forma densa e a solução é obtida por eliminação de Gauss com pivoteamento parcial. Esse método foi usado como referência para comparar os métodos iterativos.

3.2 Método de Liebmann

O método de Liebmann corresponde ao método de Gauss-Seidel aplicado às equações de diferenças finitas. Para cada nó, calcula-se uma nova temperatura a partir dos vizinhos. Os valores recém-calculados são utilizados imediatamente na mesma varredura.

O critério de convergência adotado foi:

$$\varepsilon^{(k)} = \frac{\max |T_{i,j}^{(k+1)} - T_{i,j}^{(k)}|}{\max |T_{i,j}^{(k+1)}|}. \quad (11)$$

3.3 Liebmann com relaxação

A versão relaxada utiliza:

$$T^{(k+1)} = (1 - \omega)T^{(k)} + \omega T^{(calculado)}. \quad (12)$$

Para $\omega = 1$, recupera-se o método sem relaxação. Para $1 < \omega < 2$, tem-se sobre-relaxação sucessiva. Essa estratégia pode reduzir significativamente o número de iterações necessárias para a convergência.

4 Solução analítica unidimensional de comparação

A comparação é feita ao longo da linha média da aleta. A solução analítica unidimensional clássica é expressa em termos da temperatura adimensional:

$$\theta(x) = \frac{T(x) - T_\infty}{T_b - T_\infty}. \quad (13)$$

Para uma aleta de seção transversal constante com extremidade convectiva:

$$\theta(x) = \frac{\cosh[m(L - x)] + \frac{h}{mk} \sinh[m(L - x)]}{\cosh(mL) + \frac{h}{mk} \sinh(mL)}, \quad (14)$$

em que:

$$m = \sqrt{\frac{hP}{kA_c}}. \quad (15)$$

No programa, considera-se largura unitária fora do plano. Assim, a comparação unidimensional representa uma aproximação compatível com a troca convectiva pelas faces superior e inferior do modelo bidimensional.

5 Arquivos de saída gerados pelo programa

Tabela 1: Arquivos gerados pelo programa.

Arquivo	Conteúdo
temperature_gauss.dat	Campo de temperatura por eliminação de Gauss
temperature_liebmann.dat	Campo de temperatura por Liebmann sem relaxação
temperature_sor.dat	Campo de temperatura por Liebmann com relaxação
linha_central.dat	Comparação numérica 2D e analítica 1D
estudo_relaxacao.dat	Influência de ω na convergência
estudo_malha.dat	Refinamento de malha
resumo_resultados.txt	Resumo dos resultados principais
mapa_temperatura.png	Mapa bidimensional de temperatura
contornos_isotermicos.png	Curvas de nível da temperatura
linha_central.png	Perfil térmico na linha central

6 Caso numérico de demonstração

Para demonstrar o funcionamento do programa, adotou-se o seguinte conjunto de parâmetros:

Tabela 2: Parâmetros usados na execução de referência.

Grandeza	Valor	Unidade
Comprimento da aleta, L	0,12	m
Espessura da aleta, H	0,02	m
Condutividade térmica, k	205	W/(m.K)
Coeficiente convectivo, h	25	W/(m ² .K)
Temperatura da base, T_b	100	°C
Temperatura ambiente, T_∞	25	°C
Nós na direção x	21	—
Nós na direção y	9	—
Tolerância	10^{-8}	—
Fator de relaxação	1,60	—

7 Resultados dos métodos

A Tabela 3 compara os resultados obtidos pelos três métodos implementados.

Tabela 3: Comparação entre os métodos implementados.

Método	Iterações	Erro final	Tempo [s]
Eliminação de Gauss	–	–	$1,02 \times 10^{-1}$
Liebmann sem relaxação	11012	$9,996 \times 10^{-9}$	3,93
Liebmann com relaxação, $\omega = 1,60$	3097	$9,964 \times 10^{-9}$	1,11

A diferença máxima entre a solução direta por Gauss e a solução por Liebmann sem relaxação foi aproximadamente $1,01 \times 10^{-3}$ °C. Para o método com relaxação, a diferença máxima em relação à solução direta foi aproximadamente $2,51 \times 10^{-4}$ °C.

O erro percentual médio entre a solução bidimensional na linha central e a solução analítica unidimensional foi:

$$\bar{e} = 0,0218\%. \quad (16)$$

8 Campo de temperatura obtido

A Figura 1 apresenta o mapa bidimensional de temperatura. Observa-se que a maior temperatura ocorre na base prescrita, enquanto a temperatura diminui na direção da extremidade livre devido à troca convectiva com o ambiente.

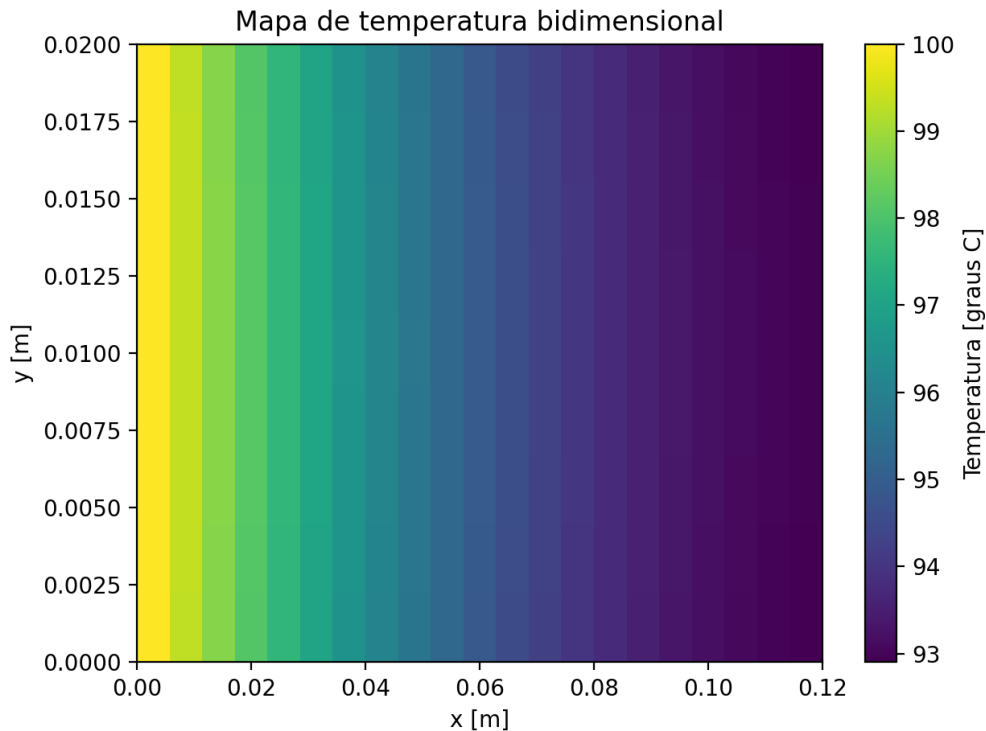


Figura 1: Mapa bidimensional de temperatura obtido pelo método de Liebmann com relaxação.

A Figura 2 apresenta os contornos isotérmicos. O formato das curvas indica a influência simultânea da condução axial e da perda de calor pelas superfícies superior, inferior e extremidade livre.

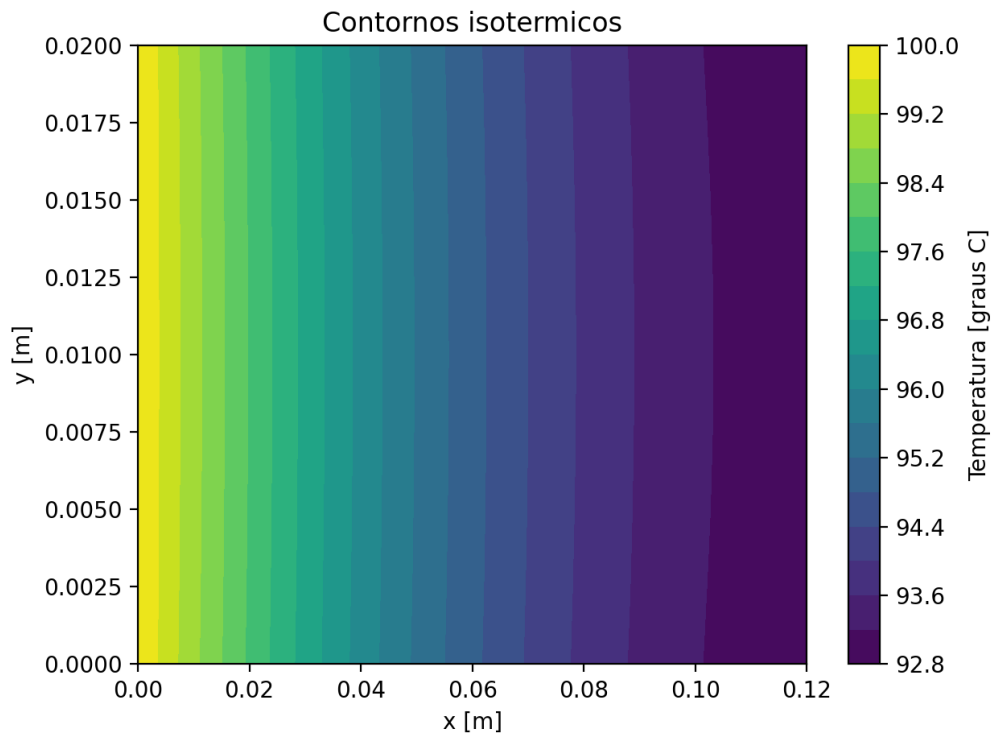


Figura 2: Curvas de nível do campo de temperatura.

9 Comparação com a solução analítica 1D

A Figura 3 compara a temperatura obtida na linha central da solução bidimensional com a solução analítica unidimensional clássica.

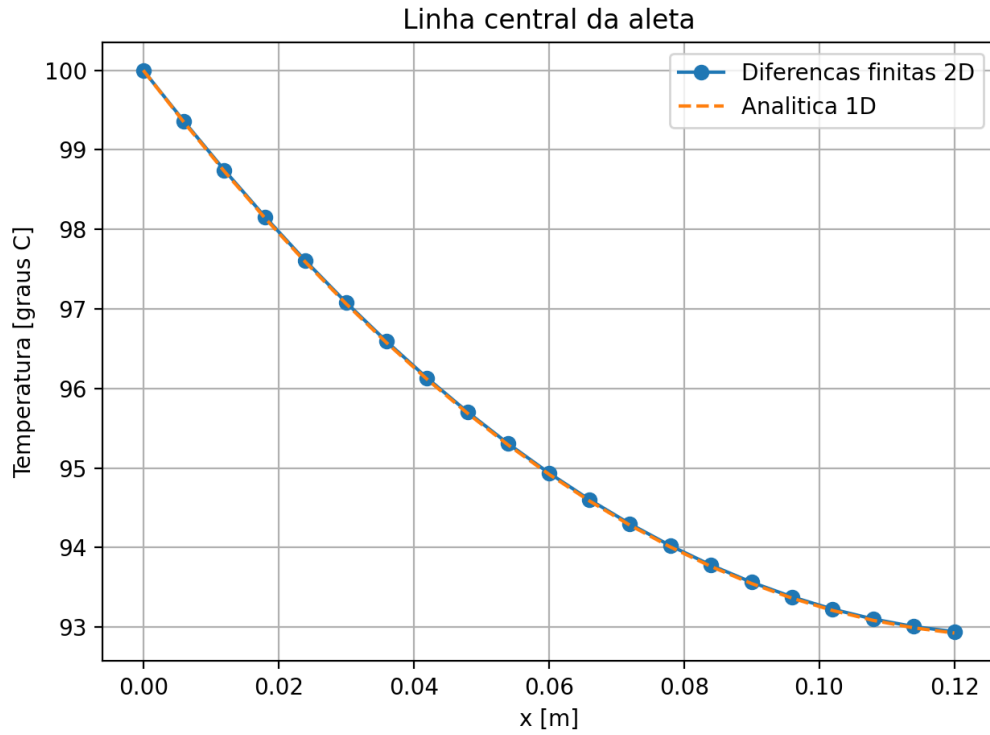


Figura 3: Comparação entre solução numérica 2D e solução analítica 1D ao longo da linha central.

A concordância é elevada para os parâmetros adotados. As diferenças observadas são esperadas, pois o modelo bidimensional representa variações na direção transversal y , enquanto a solução analítica unidimensional assume temperatura uniforme na seção transversal da aleta.

10 Estudo do fator de relaxação

A Tabela 4 apresenta o efeito do fator de relaxação na velocidade de convergência.

Tabela 4: Influência do fator de relaxação ω .

ω	Iterações	Erro final	Tempo [s]
0,8	15903	$9,995 \times 10^{-9}$	5,68
1,0	11012	$9,996 \times 10^{-9}$	3,94
1,2	7614	$9,995 \times 10^{-9}$	2,74
1,4	5084	$9,994 \times 10^{-9}$	1,82
1,6	3097	$9,964 \times 10^{-9}$	1,11
1,8	1449	$9,924 \times 10^{-9}$	0,517
1,9	678	$9,869 \times 10^{-9}$	0,243

Para o caso analisado, o aumento de ω reduziu expressivamente o número de iterações. Entretanto, valores muito próximos de 2 podem tornar o método instável em outros

problemas ou malhas, de modo que a escolha do fator de relaxação deve ser feita com cuidado.

11 Estudo de refinamento de malha

A Tabela 5 mostra o efeito do refinamento de malha no erro médio em relação à solução analítica unidimensional.

Tabela 5: Estudo de refinamento de malha.

n_x	n_y	Iterações	Erro final	Erro médio [%]
11	5	855	$9,962 \times 10^{-9}$	0,0235
21	9	3097	$9,964 \times 10^{-9}$	0,0218
31	13	6514	$9,999 \times 10^{-9}$	0,0213

O refinamento de malha reduziu levemente o erro médio em relação à solução analítica, indicando convergência espacial. Ao mesmo tempo, o custo computacional aumentou, principalmente nos métodos iterativos, pois há mais nós e mais equações acopladas.

12 Discussão dos resultados

Os três métodos convergiram para soluções praticamente equivalentes. A eliminação de Gauss foi rápida para a malha adotada, pois o número de nós ainda é pequeno. Entretanto, esse método tende a se tornar menos eficiente para malhas grandes, principalmente porque a matriz de diferenças finitas é esparsa e o armazenamento denso desperdiça memória.

O método de Liebmann sem relaxação apresentou convergência mais lenta, exigindo 11012 iterações no caso de referência. A introdução da relaxação reduziu esse número para 3097 iterações com $\omega = 1,60$. No estudo paramétrico, fatores de relaxação maiores reduziram ainda mais o número de iterações, mas a escolha de ω deve considerar estabilidade e robustez.

A comparação com o modelo analítico unidimensional apresentou erro percentual médio baixo. Essa proximidade ocorre porque a geometria escolhida é relativamente fina e a condução axial domina o comportamento global. Ainda assim, o modelo bidimensional é mais geral, pois permite observar gradientes transversais e efeitos de canto que não existem no modelo 1D.

13 Conclusão

O programa desenvolvido atendeu aos requisitos do PPC6 ao implementar a solução bidimensional da equação de Laplace para uma aleta retangular com base prescrita e superfícies convectivas. A formulação por diferenças finitas permitiu montar o sistema linear correspondente, e os métodos de Gauss, Liebmann e Liebmann com relaxação foram comparados em termos de iterações, erro e tempo computacional.

Os resultados indicam que a relaxação acelera significativamente a convergência do método iterativo. O refinamento de malha melhora gradualmente a concordância com a solução analítica unidimensional, embora aumente o custo computacional. As diferenças

entre o modelo 2D e a solução 1D são fisicamente coerentes, pois a solução bidimensional incorpora a influência das condições convectivas sobre todo o campo de temperaturas.

14 Código-fonte Python

O código completo do programa é apresentado a seguir. Ele foi escrito com comentários detalhados para explicitar a montagem da malha, a discretização por diferenças finitas, a montagem do sistema linear, a eliminação de Gauss, o método de Liebmann, a relaxação e as rotinas de pós-processamento.

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  """
5  PPC6 - Conducao de calor bidimensional em uma aleta retangular.
6
7  Este programa resolve numericamente o problema de conducao de calor em regime
8  permanente em uma aleta retangular de secao transversal constante. A base da
9  aleta possui temperatura prescrita e as demais superficies trocam calor por
10  conveccao com o ambiente.
11
12  Equacao governante:
13
14      
$$d^2T/dx^2 + d^2T/dy^2 = 0$$

15
16  Condicoes de contorno:
17
18      x = 0      -> T = T_b                      (Dirichlet)
19      x = L      -> -k dT/dn = h (T - T_inf)      (Robin)
20      y = 0      -> -k dT/dn = h (T - T_inf)      (Robin)
21      y = H      -> -k dT/dn = h (T - T_inf)      (Robin)
22
23  Metodos implementados:
24
25      1. Montagem do sistema linear por diferencas finitas;
26      2. Eliminacao de Gauss com pivoteamento parcial;
27      3. Metodo de Liebmann, isto e, Gauss-Seidel sem relaxacao;
28      4. Metodo de Liebmann com relaxacao, isto e, SOR;
29      5. Estudo do fator de relaxacao omega;
30      6. Estudo de refinamento de malha;
31      7. Comparacao da linha central com a solucao analitica unidimensional.
32
33  O codigo foi escrito de forma propositalmente didatica e comentada. A solucao
34  numerica, a montagem do sistema e os metodos iterativos foram implementados com
35  estruturas basicas de Python. Nao e utilizado numpy.linalg.solve ou qualquer
36  rotina pronta de solucao de sistemas lineares.
37
38  Autor: Mateus Leal Silva
39  Disciplina: Calculo Numerico Aplicado
40  """
41
42  from __future__ import annotations
43
44  import argparse
45  import math
46  import time
47  from dataclasses import dataclass
48  from pathlib import Path
49  from typing import Optional
50
51
52  # =====
53  # Estruturas de dados
54  # =====
55
56  @dataclass
57  class ProblemData:
58      """Agrupa os parametros fisicos e numericos do problema.
59
60      A vantagem de usar uma estrutura como esta e evitar passar muitas variaveis
61      soltas entre as funcoes. Isso reduz erros de ordem de argumentos e torna o
62      codigo mais legivel.
63      """
64
```

```

65     length: float          # comprimento da aleta, L [m]
66     thickness: float       # espessura/altura da aleta, H [m]
67     conductivity: float    # condutividade termica, k [W/(m.K)]
68     convection: float      # coeficiente convectivo, h [W/(m2.K)]
69     base_temperature: float # temperatura prescrita na base, T_b [graus C]
70     ambient_temperature: float # temperatura ambiente, T_inf [graus C]
71     nx: int                # numero de nos na direcao x
72     ny: int                # numero de nos na direcao y
73     tolerance: float       # tolerancia de convergencia dos metodos iterativos
74     omega: float           # fator de relaxacao do metodo SOR
75     max_iterations: int     # limite maximo de iteracoes
76
77
78 @dataclass
79 class IterativeResult:
80     """Armazena a resposta de um metodo iterativo."""
81
82     temperature: list[list[float]] # matriz T[j][i]
83     iterations: int                 # numero de iteracoes executadas
84     final_error: float              # erro final calculado
85     elapsed_time: float             # tempo de execucao [s]
86     converged: bool                 # indica se o criterio de convergencia foi atendido
87
88
89 @dataclass
90 class DirectResult:
91     """Armazena a resposta da eliminacao de Gauss."""
92
93     temperature: list[list[float]] # matriz T[j][i]
94     elapsed_time: float             # tempo de execucao [s]
95
96
97 # =====
98 # Funcoes auxiliares de malha e indexacao
99 # =====
100
101 def global_index(i: int, j: int, nx: int) -> int:
102     """Converte o indice bidimensional (i, j) para indice global p.
103
104     O programa utiliza a numeracao por linhas horizontais, com j variando de
105     baixo para cima e i variando da esquerda para a direita:
106
107         p = j*nx + i
108
109     Como Python usa indices iniciando em zero, o primeiro no possui p = 0.
110     Para apresentar em relatorios, basta usar p + 1.
111     """
112     return j * nx + i
113
114
115 def inverse_index(p: int, nx: int) -> tuple[int, int]:
116     """Converte o indice global p de volta para os indices (i, j)."""
117     j = p // nx
118     i = p % nx
119     return i, j
120
121
122 def mesh_spacing(data: ProblemData) -> tuple[float, float]:
123     """Calcula os espacamentos uniformes da malha.
124
125     Com nx nos na direcao x, existem nx - 1 intervalos. Assim:
126
127         dx = L/(nx - 1)
128
129     Analogamente:
130
131         dy = H/(ny - 1)
132     """
133     dx = data.length / (data.nx - 1)
134     dy = data.thickness / (data.ny - 1)
135     return dx, dy
136
137

```

```

138 def create_coordinate_vectors(data: ProblemData) -> tuple[list[float], list[float]]:
139     """Gera os vetores de coordenadas x e y da malha."""
140     dx, dy = mesh_spacing(data)
141     x_values = [i * dx for i in range(data.nx)]
142     y_values = [j * dy for j in range(data.ny)]
143     return x_values, y_values
144
145
146 def create_initial_temperature_field(data: ProblemData) -> list[list[float]]:
147     """Cria um campo inicial para os metodos iterativos.
148
149     A base e iniciada diretamente com T_b. Os demais pontos sao iniciados com
150     uma interpolacao linear simples entre T_b e T_inf ao longo de x. Isso nao e
151     obrigatorio, mas normalmente reduz um pouco o numero de iteracoes quando
152     comparado com iniciar todo o dominio em T_inf.
153     """
154     temperature: list[list[float]] = []
155
156     for j in range(data.ny):
157         row: list[float] = []
158         for i in range(data.nx):
159             if i == 0:
160                 row.append(data.base_temperature)
161             else:
162                 fraction = i / (data.nx - 1)
163                 value = (
164                     data.base_temperature
165                     + fraction * (data.ambient_temperature - data.base_temperature)
166                 )
167                 row.append(value)
168             temperature.append(row)
169
170     return temperature
171
172
173 # =====
174 # Montagem do sistema linear [A]{T} = {b}
175 # =====
176
177 def equation_coefficients_for_node(
178     i: int,
179     j: int,
180     data: ProblemData,
181 ) -> tuple[dict[int, float], float]:
182     """Monta a equacao discreta de um unico no da malha.
183
184     Retorna um dicionario de coeficientes da linha da matriz A e o termo
185     independente correspondente da matriz b.
186
187     A equacao de Laplace discretizada para um no interno e:
188
189         
$$(T_E - 2T_P + T_W)/dx^2 + (T_N - 2T_P + T_S)/dy^2 = 0$$

190
191     reorganizando para diagonal positiva:
192
193         
$$(2/dx^2 + 2/dy^2)T_P - (1/dx^2)T_E - (1/dx^2)T_W - (1/dy^2)T_N - (1/dy^2)T_S = 0$$

194
195     Nos contornos convectivos, utiliza-se um no ficticio para aplicar a condicao
196     de Robin:
197
198         
$$-k \, dT/dn = h(T - T_{inf})$$

199
200     A substituicao do no ficticio gera os coeficientes modificados das bordas.
201     """
202     nx = data.nx
203     ny = data.ny
204     dx, dy = mesh_spacing(data)
205
206     # Coeficientes basicos das derivadas segundas.
207     ax = 1.0 / (dx * dx)
208     ay = 1.0 / (dy * dy)

```

```

211
212 # Coeficientes associados aos termos convectivos que aparecem apos a
213 # eliminacao dos nos ficticios.
214 bx = data.convection / (data.conductivity * dx)
215 by = data.convection / (data.conductivity * dy)
216
217 p = global_index(i, j, nx)
218
219 # -----
220 # Condicao de Dirichlet na base da aleta: T = T_b.
221 # A linha da matriz fica simplesmente: 1*T_p = T_b.
222 # -----
223 if i == 0:
224     return {p: 1.0}, data.base_temperature
225
226 coefficients: dict[int, float] = {}
227 rhs = 0.0
228
229 def add_coefficient(node_i: int, node_j: int, value: float) -> None:
230     """Soma um coeficiente na posicao correta da linha da matriz."""
231     q = global_index(node_i, node_j, nx)
232     coefficients[q] = coefficients.get(q, 0.0) + value
233
234 # -----
235 # Contribuicao da direcao x.
236 # -----
237 if i == nx - 1:
238     # Extremidade livre x = L: contorno convectivo.
239     # 0 no ficticio externo e eliminado usando a condicao de Robin.
240     add_coefficient(i, j, 2.0 * ax + 2.0 * bx)
241     add_coefficient(i - 1, j, -2.0 * ax)
242     rhs += 2.0 * bx * data.ambient_temperature
243 else:
244     # No nao esta na extremidade direita; usa vizinhos leste e oeste.
245     add_coefficient(i, j, 2.0 * ax)
246     add_coefficient(i - 1, j, -ax)
247     add_coefficient(i + 1, j, -ax)
248
249 # -----
250 # Contribuicao da direcao y.
251 # -----
252 if j == 0:
253     # Superficie inferior: contorno convectivo.
254     add_coefficient(i, j, 2.0 * ay + 2.0 * by)
255     add_coefficient(i, j + 1, -2.0 * ay)
256     rhs += 2.0 * by * data.ambient_temperature
257 elif j == ny - 1:
258     # Superficie superior: contorno convectivo.
259     add_coefficient(i, j, 2.0 * ay + 2.0 * by)
260     add_coefficient(i, j - 1, -2.0 * ay)
261     rhs += 2.0 * by * data.ambient_temperature
262 else:
263     # No nao esta nas superficies superior ou inferior.
264     add_coefficient(i, j, 2.0 * ay)
265     add_coefficient(i, j - 1, -ay)
266     add_coefficient_north = -ay
267     add_coefficient(i, j + 1, add_coefficient_north)
268
269 return coefficients, rhs
270
271
272 def build_linear_system(data: ProblemData) -> tuple[list[list[float]], list[float]]:
273     """Monta a matriz densa A e o vetor b do sistema linear.
274
275     O armazenamento denso e usado para fins didaticos e para permitir a
276     implementacao direta da eliminacao de Gauss. Para malhas muito refinadas,
277     a matriz possui muitos zeros e o ideal seria usar armazenamento esparsa.
278     """
279     number_of_nodes = data.nx * data.ny
280
281     # Cria matriz A inicialmente preenchida por zeros.
282     matrix = [[0.0 for _ in range(number_of_nodes)] for _ in range(number_of_nodes)]
283

```

```

284 # Cria vetor b inicialmente preenchido por zeros.
285 rhs = [0.0 for _ in range(number_of_nodes)]
286
287 for j in range(data.ny):
288     for i in range(data.nx):
289         p = global_index(i, j, data.nx)
290         coefficients, independent_term = equation_coefficients_for_node(i, j, data)
291
292         for q, coefficient in coefficients.items():
293             matrix[p][q] = coefficient
294
295         rhs[p] = independent_term
296
297     return matrix, rhs
298
299
300 # =====
301 # Eliminacao de Gauss com pivoteamento parcial
302 # =====
303
304 def gaussian_elimination(matrix: list[list[float]], rhs: list[float]) -> list[float]:
305     """Resolve  $[A]\{x\} = \{b\}$  por eliminacao de Gauss.
306
307     Esta implementacao utiliza pivoteamento parcial. Em cada coluna, procura-se
308     a linha com o maior valor absoluto do coeficiente pivô. Isso melhora a
309     estabilidade numerica e evita divisao por valores muito pequenos quando uma
310     troca de linhas resolve o problema.
311     """
312     n = len(rhs)
313
314     # Copias sao criadas para nao modificar os objetos originais.
315     a = [row[:] for row in matrix]
316     b = rhs[:]
317
318     # -----
319     # Etapa de eliminacao progressiva: transforma A em matriz triangular superior.
320     # -----
321     for pivot_column in range(n - 1):
322         # Procura o melhor pivô na coluna atual.
323         pivot_row = pivot_column
324         pivot_abs_value = abs(a[pivot_column][pivot_column])
325
326         for candidate_row in range(pivot_column + 1, n):
327             candidate_abs_value = abs(a[candidate_row][pivot_column])
328             if candidate_abs_value > pivot_abs_value:
329                 pivot_abs_value = candidate_abs_value
330                 pivot_row = candidate_row
331
332         if pivot_abs_value < 1.0e-20:
333             raise ZeroDivisionError(
334                 "Pivô numericamente nulo encontrado na eliminacao de Gauss."
335             )
336
337         # Troca as linhas, se necessario.
338         if pivot_row != pivot_column:
339             a[pivot_column], a[pivot_row] = a[pivot_row], a[pivot_column]
340             b[pivot_column], b[pivot_row] = b[pivot_row], b[pivot_column]
341
342         pivot = a[pivot_column][pivot_column]
343
344         # Zera os coeficientes abaixo do pivô.
345         for row in range(pivot_column + 1, n):
346             factor = a[row][pivot_column] / pivot
347
348             # O coeficiente abaixo do pivô e definido explicitamente como zero.
349             a[row][pivot_column] = 0.0
350
351             # Atualiza os demais coeficientes da linha.
352             for column in range(pivot_column + 1, n):
353                 a[row][column] -= factor * a[pivot_column][column]
354
355             # Atualiza tambem o lado direito.
356             b[row] -= factor * b[pivot_column]

```

```

357
358 # -----
359 # Etapa de substituicao regressiva: resolve de baixo para cima.
360 # -----
361 x = [0.0 for _ in range(n)]
362
363 for row in range(n - 1, -1, -1):
364     summation = 0.0
365
366     for column in range(row + 1, n):
367         summation += a[row][column] * x[column]
368
369     diagonal = a[row][row]
370     if abs(diagonal) < 1.0e-20:
371         raise ZeroDivisionError(
372             "Diagonal numericamente nula encontrada na substituicao regressiva."
373         )
374
375     x[row] = (b[row] - summation) / diagonal
376
377 return x
378
379
380 def vector_to_temperature_field(vector: list[float], nx: int, ny: int) -> list[list[float]]:
381     """Converte a solucao vetorial para uma matriz T[j][i]."""
382     temperature = [[0.0 for _ in range(nx)] for _ in range(ny)]
383
384     for p, value in enumerate(vector):
385         i, j = inverse_index(p, nx)
386         temperature[j][i] = value
387
388     return temperature
389
390
391 def solve_by_gauss(data: ProblemData) -> DirectResult:
392     """Monta e resolve o sistema linear pelo metodo direto de Gauss."""
393     start = time.perf_counter()
394     matrix, rhs = build_linear_system(data)
395     solution_vector = gaussian_elimination(matrix, rhs)
396     elapsed = time.perf_counter() - start
397
398     temperature = vector_to_temperature_field(solution_vector, data.nx, data.ny)
399     return DirectResult(temperature=temperature, elapsed_time=elapsed)
400
401
402 # =====
403 # Metodos iterativos de Liebmann e SOR
404 # =====
405
406 def local_update_value(
407     i: int,
408     j: int,
409     temperature: list[list[float]],
410     data: ProblemData,
411 ) -> float:
412     """Calcula o valor de T no no (i, j) usando a equacao local.
413
414     Esta funcao usa a mesma discretizacao da matriz global, mas sem montar a
415     matriz. Ela e conveniente para os metodos de Gauss-Seidel e SOR.
416
417     A equacao geral de uma linha pode ser escrita como:
418
419         a_P T_P - soma(a_vizinho T_vizinho) = b
420
421     Logo:
422
423         T_P = [soma(a_vizinho T_vizinho) + b]/a_P
424     """
425     # A base prescrita nao deve ser atualizada iterativamente.
426     if i == 0:
427         return data.base_temperature
428

```



```

429 coefficients, rhs = equation_coefficients_for_node(i, j, data)
430 p = global_index(i, j, data.nx)
431 diagonal = coefficients[p]
432
433 neighbor_sum = 0.0
434
435 for q, coefficient in coefficients.items():
436     if q == p:
437         continue
438
439     neighbor_i, neighbor_j = inverse_index(q, data.nx)
440
441     # Como os coeficientes dos vizinhos aparecem negativos na matriz, eles
442     # são levados para o outro lado da equação com sinal positivo.
443     neighbor_sum += (-coefficient) * temperature[neighbor_j][neighbor_i]
444
445 return (neighbor_sum + rhs) / diagonal
446
447 def solve_by_liebmann(data: ProblemData, omega: float) -> IterativeResult:
448     """Resolve o problema por Liebmann/Gauss-Seidel com relaxação opcional.
449
450     Quando omega = 1, tem-se o método de Liebmann sem relaxação.
451     Quando 1 < omega < 2, tem-se sobre-relaxação sucessiva (SOR).
452     Quando 0 < omega < 1, tem-se sub-relaxação.
453
454     A atualização adotada é:
455
456         T_novo = (1 - omega) T_velho + omega T_calculado
457
458     em que T_calculado é o valor obtido diretamente pela equação de diferenças.
459     """
460
461     temperature = create_initial_temperature_field(data)
462
463     start = time.perf_counter()
464     final_error = math.inf
465     converged = False
466
467     for iteration in range(1, data.max_iterations + 1):
468         maximum_change = 0.0
469         maximum_reference = 1.0e-12
470
471         # Varre todos os nós da malha. A varredura linha a linha caracteriza o
472         # uso imediato dos valores já atualizados, como no Gauss-Seidel.
473         for j in range(data.ny):
474             for i in range(data.nx):
475                 old_value = temperature[j][i]
476
477                 calculated_value = local_update_value(i, j, temperature, data)
478                 new_value = (1.0 - omega) * old_value + omega * calculated_value
479
480                 # Garante novamente a condição de Dirichlet da base.
481                 if i == 0:
482                     new_value = data.base_temperature
483
484                 temperature[j][i] = new_value
485
486                 change = abs(new_value - old_value)
487                 maximum_change = max(maximum_change, change)
488                 maximum_reference = max(maximum_reference, abs(new_value))
489
490             final_error = maximum_change / maximum_reference
491
492             if final_error < data.tolerance:
493                 converged = True
494                 break
495
496     elapsed = time.perf_counter() - start
497
498     return IterativeResult(
499         temperature=temperature,
500         iterations=iteration,
501         final_error=final_error,

```

```

502         elapsed_time=elapsed,
503         converged=converged,
504     )
505
506
507 # =====
508 # Comparacoes, pos-processamento e solucao analitica 1D
509 # =====
510
511 def max_abs_difference(
512     first: list[list[float]],
513     second: list[list[float]],
514 ) -> float:
515     """Calcula a maior diferenca absoluta entre dois campos de temperatura."""
516     maximum = 0.0
517
518     for row_a, row_b in zip(first, second):
519         for value_a, value_b in zip(row_a, row_b):
520             maximum = max(maximum, abs(value_a - value_b))
521
522     return maximum
523
524
525 def interpolate_temperature_at_y(
526     temperature: list[list[float]],
527     y_target: float,
528     data: ProblemData,
529 ) -> list[float]:
530     """Extrai T(x, y_target) por interpolacao linear na direcao y."""
531     _, y_values = create_coordinate_vectors(data)
532
533     if y_target <= y_values[0]:
534         return temperature[0][:]
535
536     if y_target >= y_values[-1]:
537         return temperature[-1][:]
538
539     for j in range(1, data.ny):
540         if y_values[j - 1] <= y_target <= y_values[j]:
541             y0 = y_values[j - 1]
542             y1 = y_values[j]
543             fraction = (y_target - y0) / (y1 - y0)
544
545             line = []
546             for i in range(data.nx):
547                 value = temperature[j - 1][i] * (1.0 - fraction) + temperature[j][i] *
fraction
548                 line.append(value)
549
550             return line
551
552     # Por seguranca, retorna a ultima linha se algo numericamente inesperado ocorrer.
553     return temperature[-1][:]
554
555
556 def analytical_1d_temperature(x: float, data: ProblemData) -> float:
557     """Calcula a solucao analitica 1D classica da aleta com ponta convectiva.
558
559     A forma adimensional e:
560
561         
$$\theta(x) = \frac{\cosh(m(L-x)) + h/(m k) \sinh(m(L-x))}{\cosh(mL) + h/(m k) \sinh(mL)}$$

562
563     em que:
564
565         
$$\theta = (T - T_{\text{inf}})/(T_b - T_{\text{inf}})$$

566         
$$m = \sqrt{h P/(k A_c)}$$

567
568     Para compatibilizar com o modelo bidimensional usado aqui, considera-se
569     largura unitaria na direcao fora do plano e convecção pelas faces superior e
570     inferior. Assim:
571
572         
$$P = 2 * \text{largura\_unitaria}$$

573

```

```

574     A_c = H * largura_unitaria
575
576     Logo P/A_c = 2/H.
577     """
578     unit_width = 1.0
579     perimeter = 2.0 * unit_width
580     area = data.thickness * unit_width
581
582     m = math.sqrt(data.convection * perimeter / (data.conductivity * area))
583
584     if m == 0.0:
585         return data.base_temperature
586
587     numerator = (
588         math.cosh(m * (data.length - x))
589         + data.convection / (m * data.conductivity)
590         * math.sinh(m * (data.length - x))
591     )
592
593     denominator = (
594         math.cosh(m * data.length)
595         + data.convection / (m * data.conductivity)
596         * math.sinh(m * data.length)
597     )
598
599     theta = numerator / denominator
600     temperature = data.ambient_temperature + theta * (
601         data.base_temperature - data.ambient_temperature
602     )
603
604     return temperature
605
606
607 def central_line_comparison(
608     temperature: list[list[float]],
609     data: ProblemData,
610 ) -> tuple[list[tuple[float, float, float, float]], float]:
611     """Compara T numerico na linha central com a solucao analitica 1D.
612
613     Retorna uma lista com:
614
615         x, T_numerico, T_analitico, erro_percentual
616
617     e tambem o erro percentual medio.
618     """
619     x_values, _ = create_coordinate_vectors(data)
620     y_mid = 0.5 * data.thickness
621     numerical_line = interpolate_temperature_at_y(temperature, y_mid, data)
622
623     rows: list[tuple[float, float, float, float]] = []
624     errors = []
625
626     for x, numerical_temperature in zip(x_values, numerical_line):
627         analytical_temperature = analytical_1d_temperature(x, data)
628
629         theta_numerical = (
630             (numerical_temperature - data.ambient_temperature)
631             / (data.base_temperature - data.ambient_temperature)
632         )
633         theta_analytical = (
634             (analytical_temperature - data.ambient_temperature)
635             / (data.base_temperature - data.ambient_temperature)
636         )
637
638         denominator = max(abs(theta_analytical), 1.0e-12)
639         percent_error = abs(theta_numerical - theta_analytical) / denominator * 100.0
640
641         rows.append((x, numerical_temperature, analytical_temperature, percent_error))
642         errors.append(percent_error)
643
644     mean_error = sum(errors) / len(errors)
645     return rows, mean_error
646

```

```

647
648 # =====
649 # Estudos numericos exigidos no PPC6
650 # =====
651
652 def relaxation_study(data: ProblemData, omegas: list[float]) -> list[tuple[float, int,
float, float, bool]]:
653     """Executa o estudo do efeito do fator de relaxacao omega."""
654     results = []
655
656     for omega in omegas:
657         result = solve_by_liebmann(data, omega=omega)
658         results.append(
659             (
660                 omega,
661                 result.iterations,
662                 result.final_error,
663                 result.elapsed_time,
664                 result.converged,
665             )
666         )
667
668     return results
669
670
671 def mesh_refinement_study(
672     base_data: ProblemData,
673     meshes: list[tuple[int, int]],
674 ) -> list[tuple[int, int, int, float, float, bool]]:
675     """Executa o estudo de refinamento de malha.
676
677     Para cada malha, resolve-se o problema com SOR e calcula-se o erro percentual
678     medio da linha central em relacao ao modelo analitico 1D.
679     """
680     results = []
681
682     for nx, ny in meshes:
683         data = ProblemData(
684             length=base_data.length,
685             thickness=base_data.thickness,
686             conductivity=base_data.conductivity,
687             convection=base_data.convection,
688             base_temperature=base_data.base_temperature,
689             ambient_temperature=base_data.ambient_temperature,
690             nx=nx,
691             ny=ny,
692             tolerance=base_data.tolerance,
693             omega=base_data.omega,
694             max_iterations=base_data.max_iterations,
695         )
696
697         result = solve_by_liebmann(data, omega=data.omega)
698         _, mean_error = central_line_comparison(result.temperature, data)
699
700         results.append(
701             (
702                 nx,
703                 ny,
704                 result.iterations,
705                 result.final_error,
706                 result.elapsed_time,
707                 mean_error,
708                 result.converged,
709             )
710         )
711
712     return results
713
714
715 # =====
716 # Escrita de arquivos de saida
717 # =====
718

```

```

719 def save_temperature_field(
720     filename: Path,
721     temperature: list[list[float]],
722     data: ProblemData,
723 ) -> None:
724     """Salva x, y e T para todos os nos da malha."""
725     x_values, y_values = create_coordinate_vectors(data)
726
727     with filename.open("w", encoding="utf-8") as file:
728         file.write("# x y T\n")
729
730         for j, y in enumerate(y_values):
731             for i, x in enumerate(x_values):
732                 file.write(f"{x:.12e} {y:.12e} {temperature[j][i]:.12e}\n")
733
734             # Linha em branco separa as linhas horizontais da malha.
735             file.write("\n")
736
737
738 def save_central_line(
739     filename: Path,
740     rows: list[tuple[float, float, float, float]],
741 ) -> None:
742     """Salva a comparacao da linha central em arquivo de texto."""
743     with filename.open("w", encoding="utf-8") as file:
744         file.write("# x T_numerico T_analitico erro_percentual\n")
745
746         for x, numerical, analytical, error in rows:
747             file.write(
748                 f"{x:.12e} {numerical:.12e} {analytical:.12e} {error:.12e}\n"
749             )
750
751
752 def save_relaxation_study(
753     filename: Path,
754     rows: list[tuple[float, int, float, float, bool]],
755 ) -> None:
756     """Salva os resultados do estudo de omega."""
757     with filename.open("w", encoding="utf-8") as file:
758         file.write("# omega iteracoes erro_final tempo_s convergiu\n")
759
760         for omega, iterations, error, elapsed, converged in rows:
761             file.write(
762                 f"{omega:.6f} {iterations:d} {error:.12e} "
763                 f"{elapsed:.12e} {int(converged):d}\n"
764             )
765
766
767 def save_mesh_refinement_study(
768     filename: Path,
769     rows: list[tuple[int, int, int, float, float, float, bool]],
770 ) -> None:
771     """Salva os resultados do estudo de refinamento de malha."""
772     with filename.open("w", encoding="utf-8") as file:
773         file.write("# nx ny iteracoes erro_final tempo_s erro_medio_percentual convergiu\n")
774
775         for nx, ny, iterations, error, elapsed, mean_error, converged in rows:
776             file.write(
777                 f"{nx:d} {ny:d} {iterations:d} {error:.12e} "
778                 f"{elapsed:.12e} {mean_error:.12e} {int(converged):d}\n"
779             )
780
781
782 def save_summary(
783     filename: Path,
784     data: ProblemData,
785     gauss: DirectResult,
786     liebmann: IterativeResult,
787     sor: IterativeResult,
788     central_line_error: float,
789 ) -> None:
790     """Salva um resumo dos resultados principais."""

```

```

791 maximum_difference_liebmman = max_abs_difference(gauss.temperature, liebmann.
temperature)
792 maximum_difference_sor = max_abs_difference(gauss.temperature, sor.temperature)
793
794 with filename.open("w", encoding="utf-8") as file:
795     file.write("Resumo do PPC6 - Aleta retangular\n")
796     file.write("=====\n\n")
797     file.write(f"L = {data.length:.6g} m\n")
798     file.write(f"H = {data.thickness:.6g} m\n")
799     file.write(f"k = {data.conductivity:.6g} W/(m.K)\n")
800     file.write(f"h = {data.convection:.6g} W/(m2.K)\n")
801     file.write(f"T_b = {data.base_temperature:.6g} graus C\n")
802     file.write(f"T_inf = {data.ambient_temperature:.6g} graus C\n")
803     file.write(f"nx = {data.nx}, ny = {data.ny}\n")
804     file.write(f"tolerancia = {data.tolerance:.3e}\n")
805     file.write(f"omega = {data.omega:.6g}\n\n")
806
807     file.write("Resultados dos metodos\n")
808     file.write("-----\n")
809     file.write(f"Gauss: tempo = {gauss.elapsed_time:.6e} s\n")
810     file.write(
811         f"Liebmman: iteracoes = {liebmann.iterations}, "
812         f"erro = {liebmann.final_error:.6e}, "
813         f"tempo = {liebmann.elapsed_time:.6e} s, "
814         f"convergiu = {liebmann.converged}\n"
815     )
816     file.write(
817         f"SOR: iteracoes = {sor.iterations}, "
818         f"erro = {sor.final_error:.6e}, "
819         f"tempo = {sor.elapsed_time:.6e} s, "
820         f"convergiu = {sor.converged}\n"
821     )
822     file.write(f"Diferenca maxima Gauss x Liebmann = {maximum_difference_liebmman:.6e}
} graus C\n")
823     file.write(f"Diferenca maxima Gauss x SOR = {maximum_difference_sor:.6e} graus C\
n")
824     file.write(f"Erro percentual medio da linha central = {central_line_error:.6e}%\n
")
825
826
827 # =====
828 # Graficos
829 # =====
830
831 def generate_plots(
832     temperature: list[list[float]],
833     central_line_rows: list[tuple[float, float, float, float]],
834     data: ProblemData,
835     output_dir: Path,
836 ) -> list[Path]:
837     """Gera os graficos exigidos no enunciado.
838
839     A biblioteca matplotlib e usada apenas para visualizacao. Se ela nao estiver
840     instalada, os arquivos de dados continuam sendo gerados normalmente.
841     """
842     try:
843         import matplotlib.pyplot as plt
844     except ImportError:
845         print("Aviso: matplotlib nao encontrado. Os graficos nao foram gerados.")
846         return []
847
848     x_values, y_values = create_coordinate_vectors(data)
849
850     # Matrices X e Y no mesmo formato de temperature[j][i].
851     x_grid = [[x for x in x_values] for _ in y_values]
852     y_grid = [[y for _ in x_values] for y in y_values]
853
854     generated_files: list[Path] = []
855
856     # -----
857     # Mapa de temperatura bidimensional.
858     # -----
859     map_path = output_dir / "mapa_temperatura.png"

```

```

860 plt.figure()
861 image = plt.imshow(
862     temperature,
863     extent=[0.0, data.length, 0.0, data.thickness],
864     origin="lower",
865     aspect="auto",
866 )
867 plt.colorbar(image, label="Temperatura [graus C]")
868 plt.xlabel("x [m]")
869 plt.ylabel("y [m]")
870 plt.title("Mapa de temperatura bidimensional")
871 plt.tight_layout()
872 plt.savefig(map_path, dpi=200)
873 plt.close()
874 generated_files.append(map_path)
875
876 # -----
877 # Curvas de nivel ou contornos isotermicos.
878 # -----
879 contour_path = output_dir / "contornos_isotermicos.png"
880 plt.figure()
881 contour = plt.contourf(x_grid, y_grid, temperature, levels=20)
882 plt.colorbar(contour, label="Temperatura [graus C]")
883 plt.xlabel("x [m]")
884 plt.ylabel("y [m]")
885 plt.title("Contornos isotermicos")
886 plt.tight_layout()
887 plt.savefig(contour_path, dpi=200)
888 plt.close()
889 generated_files.append(contour_path)
890
891 # -----
892 # Distribuicao de temperatura na linha central.
893 # -----
894 line_path = output_dir / "linha_central.png"
895 x_line = [row[0] for row in central_line_rows]
896 numerical_line = [row[1] for row in central_line_rows]
897 analytical_line = [row[2] for row in central_line_rows]
898
899 plt.figure()
900 plt.plot(x_line, numerical_line, marker="o", label="Diferencas finitas 2D")
901 plt.plot(x_line, analytical_line, linestyle="--", label="Analitica 1D")
902 plt.xlabel("x [m]")
903 plt.ylabel("Temperatura [graus C]")
904 plt.title("Linha central da aleta")
905 plt.grid(True)
906 plt.legend()
907 plt.tight_layout()
908 plt.savefig(line_path, dpi=200)
909 plt.close()
910 generated_files.append(line_path)
911
912 return generated_files
913
914 # =====
915 # Interface de entrada
916 # =====
917
918 def read_float(prompt: str, default: float) -> float:
919     """Le um numero real do terminal, aceitando virgula ou ponto decimal."""
920     text = input(f"{prompt} [{default}]: ").strip()
921
922     if not text:
923         return default
924
925     return float(text.replace(",", "."))
926
927
928 def read_int(prompt: str, default: int) -> int:
929     """Le um numero inteiro do terminal."""
930     text = input(f"{prompt} [{default}]: ").strip()
931
932

```

```

933     if not text:
934         return default
935
936     return int(text)
937
938
939 def interactive_input(default_data: ProblemData) -> ProblemData:
940     """Solicita os dados do problema ao usuario."""
941     print("PPC6 - Aleta retangular por diferencas finitas")
942     print("Pressione Enter para manter o valor padrao indicado.\n")
943
944     length = read_float("Comprimento da aleta L [m]", default_data.length)
945     thickness = read_float("Espessura/altura da aleta H [m]", default_data.thickness)
946     conductivity = read_float("Condutividade termica k [W/(m.K)]", default_data.
947                               conductivity)
948     convection = read_float("Coeficiente convectivo h [W/(m2.K)]", default_data.
949                             convection)
950     base_temperature = read_float("Temperatura da base T_b [graus C]", default_data.
951                                   base_temperature)
952     ambient_temperature = read_float("Temperatura ambiente T_inf [graus C]", default_data.
953                                       ambient_temperature)
954     nx = read_int("Numero de nos na direcao x", default_data.nx)
955     ny = read_int("Numero de nos na direcao y", default_data.ny)
956     tolerance = read_float("Tolerancia de convergencia", default_data.tolerance)
957     omega = read_float("Fator de relaxacao omega", default_data.omega)
958     max_iterations = read_int("Numero maximo de iteracoes", default_data.max_iterations)
959
960     return ProblemData(
961         length=length,
962         thickness=thickness,
963         conductivity=conductivity,
964         convection=convection,
965         base_temperature=base_temperature,
966         ambient_temperature=ambient_temperature,
967         nx=nx,
968         ny=ny,
969         tolerance=tolerance,
970         omega=omega,
971         max_iterations=max_iterations,
972     )
973
974 def validate_data(data: ProblemData) -> None:
975     """Verifica se os parametros fornecidos sao fisicamente e numericamente validos."""
976     if data.length <= 0.0:
977         raise ValueError("O comprimento L deve ser positivo.")
978     if data.thickness <= 0.0:
979         raise ValueError("A espessura H deve ser positiva.")
980     if data.conductivity <= 0.0:
981         raise ValueError("A condutividade termica k deve ser positiva.")
982     if data.convection < 0.0:
983         raise ValueError("O coeficiente convectivo h nao pode ser negativo.")
984     if data.nx < 3:
985         raise ValueError("Use pelo menos 3 nos na direcao x.")
986     if data.ny < 3:
987         raise ValueError("Use pelo menos 3 nos na direcao y.")
988     if data.tolerance <= 0.0:
989         raise ValueError("A tolerancia deve ser positiva.")
990     if not (0.0 < data.omega < 2.0):
991         raise ValueError("Para SOR, recomenda-se 0 < omega < 2.")
992     if data.max_iterations <= 0:
993         raise ValueError("O numero maximo de iteracoes deve ser positivo.")
994
995 def build_argument_parser() -> argparse.ArgumentParser:
996     """Cria a interface de linha de comando."""
997     parser = argparse.ArgumentParser(
998         description="Resolve a aleta retangular 2D por diferencas finitas."
999     )
1000     parser.add_argument("--L", type=float, default=0.12, help="comprimento da aleta [m]")
1001     parser.add_argument("--H", type=float, default=0.02, help="espessura/altura da aleta [m]")

```



```

1001     parser.add_argument("--k", type=float, default=205.0, help="condutividade termica [W
1002     parser.add_argument("--h", type=float, default=25.0, help="coeficiente convectivo [W
1003     parser.add_argument("--Tb", type=float, default=100.0, help="temperatura da base [
1004     parser.add_argument("--Tinf", type=float, default=25.0, help="temperatura ambiente [
1005     parser.add_argument("--nx", type=int, default=21, help="numero de nos na direcao x")
1006     parser.add_argument("--ny", type=int, default=9, help="numero de nos na direcao y")
1007     parser.add_argument("--tol", type=float, default=1.0e-8, help="tolerancia iterativa")
1008     parser.add_argument("--omega", type=float, default=1.60, help="fator de relaxacao SOR
1009     parser.add_argument("--max-iter", type=int, default=100000, help="maximo de iteracoes
1010     parser.add_argument("--outdir", type=Path, default=Path("resultados_ppc6"), help="
1011     parser.add_argument("--interativo", action="store_true", help="solicita os parametros
1012     parser.add_argument("--sem-plots", action="store_true", help="nao gera figuras PNG")
1013     parser.add_argument("--pular-estudos", action="store_true", help="nao executa estudos
1014
1015     return parser
1016
1017
1018 # =====
1019 # Funcao principal
1020 # =====
1021
1022 def main() -> int:
1023     """Executa o fluxo completo do PPC6."""
1024     parser = build_argument_parser()
1025     args = parser.parse_args()
1026
1027     data = ProblemData(
1028         length=args.L,
1029         thickness=args.H,
1030         conductivity=args.k,
1031         convection=args.h,
1032         base_temperature=args.Tb,
1033         ambient_temperature=args.Tinf,
1034         nx=args.nx,
1035         ny=args.ny,
1036         tolerance=args.tol,
1037         omega=args.omega,
1038         max_iterations=args.max_iter,
1039     )
1040
1041     if args.interativo:
1042         data = interactive_input(data)
1043
1044     validate_data(data)
1045     args.outdir.mkdir(parents=True, exist_ok=True)
1046
1047     print("\nMontando e resolvendo o problema da aleta retangular...")
1048     print(f"Malha: {data.nx} x {data.ny} = {data.nx * data.ny} nos")
1049
1050     # -----
1051     # Metodo direto: eliminacao de Gauss.
1052     # -----
1053     gauss = solve_by_gauss(data)
1054     print(f"Gauss concluido em {gauss.elapsed_time:.6e} s")
1055
1056     # -----
1057     # Liebmann sem relaxacao: omega = 1.
1058     # -----
1059     liebmann = solve_by_liebmann(data, omega=1.0)
1060     print(
1061         "Liebmann: "
1062         f"iteracoes = {liebmann.iterations}, "
1063         f"erro = {liebmann.final_error:.6e}, "
1064         f"tempo = {liebmann.elapsed_time:.6e} s"

```

```

1065 )
1066
1067 # -----
1068 # Liebmann com relaxacao: omega definido pelo usuario.
1069 # -----
1070 sor = solve_by_liebmann(data, omega=data.omega)
1071 print(
1072     "SOR: "
1073     f"iteracoes = {sor.iterations}, "
1074     f"erro = {sor.final_error:.6e}, "
1075     f"tempo = {sor.elapsed_time:.6e} s"
1076 )
1077
1078 # -----
1079 # Comparacao com a solucao analitica 1D na linha central.
1080 # -----
1081 central_rows, mean_error = central_line_comparison(sor.temperature, data)
1082 print(f"Erro percentual medio na linha central = {mean_error:.6f}%")
1083
1084 # -----
1085 # Escrita dos arquivos principais.
1086 # -----
1087 save_temperature_field(args.outdir / "temperature_gauss.dat", gauss.temperature, data)
1088 save_temperature_field(args.outdir / "temperature_liebmann.dat", liebmann.temperature, data)
1089 save_temperature_field(args.outdir / "temperature_sor.dat", sor.temperature, data)
1090 save_central_line(args.outdir / "linha_central.dat", central_rows)
1091 save_summary(args.outdir / "resumo_resultados.txt", data, gauss, liebmann, sor, mean_error)
1092
1093 # -----
1094 # Estudos adicionais solicitados no enunciado.
1095 # -----
1096 if not args.pular_estudos:
1097     omegas = [0.8, 1.0, 1.2, 1.4, 1.6, 1.8, 1.9]
1098     omega_rows = relaxation_study(data, omegas)
1099     save_relaxation_study(args.outdir / "estudo_relaxacao.dat", omega_rows)
1100
1101     # Malhas impares ajudam a existir uma linha central exatamente em  $y = H/2$ .
1102     mesh_rows = mesh_refinement_study(
1103         data,
1104         meshes=[(11, 5), (21, 9), (31, 13)],
1105     )
1106     save_mesh_refinement_study(args.outdir / "estudo_malha.dat", mesh_rows)
1107
1108 # -----
1109 # Figuras.
1110 # -----
1111 if not args.sem_plots:
1112     generated = generate_plots(sor.temperature, central_rows, data, args.outdir)
1113     if generated:
1114         print("Graficos gerados:")
1115         for path in generated:
1116             print(f" {path}")
1117
1118     print("\nArquivos salvos em:", args.outdir)
1119     print("Diferenca maxima Gauss x Liebmann:", f"{max_abs_difference(gauss.temperature, liebmann.temperature):.6e}")
1120     print("Diferenca maxima Gauss x SOR:", f"{max_abs_difference(gauss.temperature, sor.temperature):.6e}")
1121
1122     return 0
1123
1124
1125 if __name__ == "__main__":
1126     raise SystemExit(main())

```